

MIPS Prático e a **Ascensão do RISC-V**

Aprofundamento em Algoritmos de Assembly e Arquiteturas Modernas

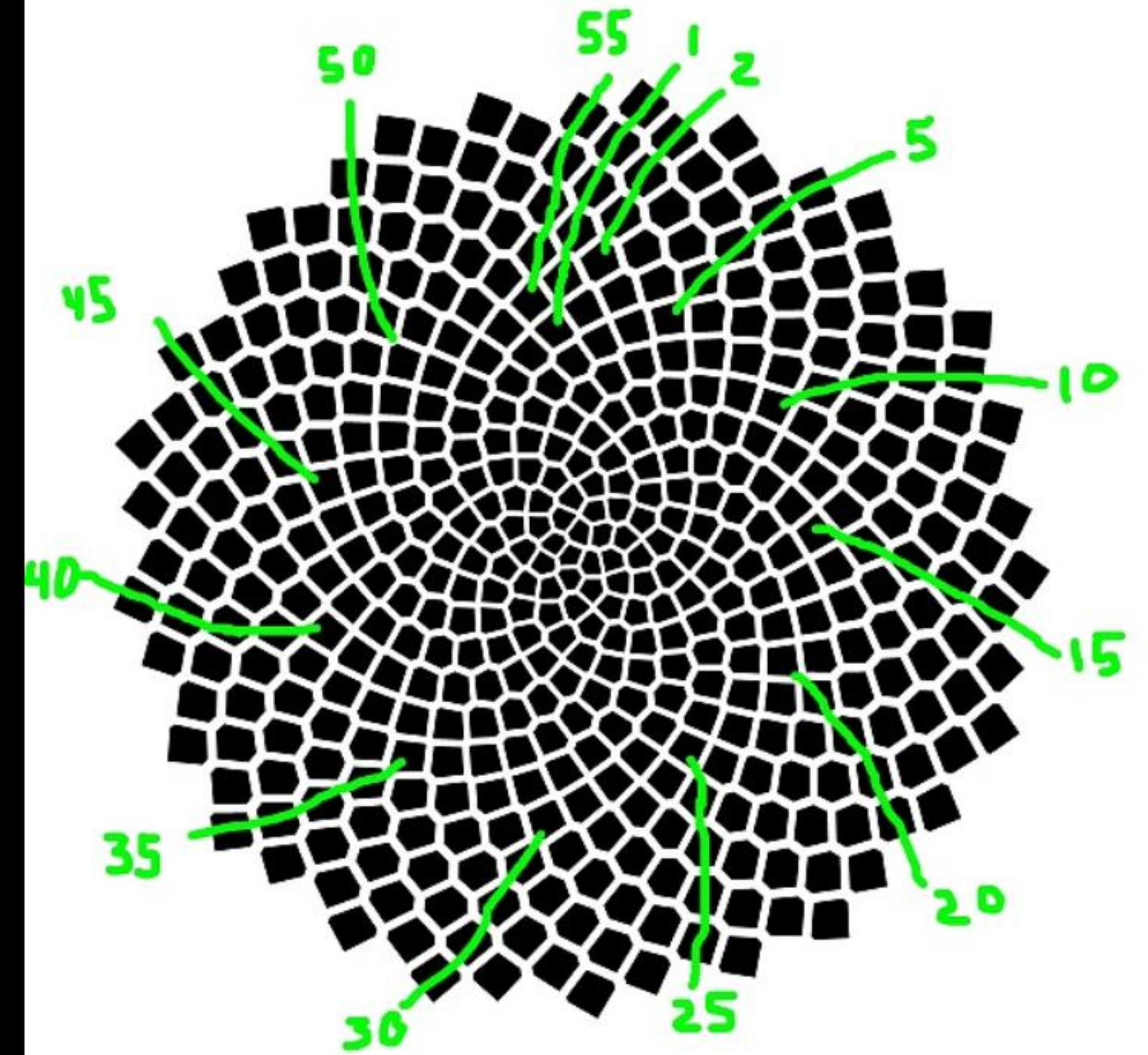
Parte 1: **MIPS Avançado**
Fundamentos de Laboratório

Teoria: Sequência de Fibonacci

A famosa sequência matemática representa um crescimento dinâmico onde cada novo elemento é gerado pela soma de seus dois predecessores imediatos:

$$F_n = F_{n-1} + F_{n-2}$$

- **Mapeamento de Casos Base:** É imperativo inicializar os primeiros passos da execução definindo estaticamente que $F_0 = 0$ e $F_1 = 1$.
- **Estratégia Computacional:** Implementaremos a abordagem iterativa, que reduz a complexidade para linear $O(n)$, economizando espaço de pilha.



Lógica: Fibonacci no **MARS**

→ Estrutura de Registradores:

- → \$t0 guarda o termo F_{n-2} (inicializado com 0)
- → \$t1 guarda o termo F_{n-1} (inicializado com 1)

→ O Loop Principal:

1. → Executa `add $t2, $t0, $t1` para calcular o termo atual.
2. → Imprime \$t2 usando a chamada de sistema `syscall` 1 (print integer).
3. → Rotaciona os dados: `move $t0, $t1` seguido de `move $t1, $t2`.

→ Terminação: Decrementa o contador e desvia via `bne`.

CÓDIGO COMENTADO (MIPS)

Loop de Fibonacci iterativo

loop:

```
add $t2, $t0, $t1 # Fn = Fn-1 + Fn-2
li  $v0, 1        # Serviço syscall: Print Int
move $a0, $t2      # Argumento passa valor
syscall           # Executa o print

move $t0, $t1      # Fn-2 assume valor anterior
move $t1, $t2      # Fn-1 assume valor atual
subi $a1, $a1, 1   # Decrementa contador
bne  $a1, $zero, loop
```


Teoria: Decimal para Hexadecimal

A base Hexadecimal (base 16) é o agrupamento prático e legível de bytes. Cada dígito hexadecimal representa um **Nibble** (exatos 4 bits), já que:

$$2^4 = 16$$

→ **Mapeamento de Caracteres ASCII:**

- Valores decimais de 0 a 9 correspondem diretamente aos caracteres '0' a '9'.
- Valores decimais de 10 a 15 traduzem-se nas letras ASCII 'A' a 'F'.

→ **Matemática de Offset:** Para gerar a saída textual correta, somamos constantes ASCII pré-definidas (48 para números e 55 para letras).

HOW TO CONVERT HEX TO BINARY?

- 1 Convert each hexadecimal digit to 4-bit binary
- 2 Combine all the binary digits
- 3 Obtain the binary equivalent of the hexadecimal number

Hex
3A

00111010

Binary

BENEFITS OF USING HEX TO BINARY CONVERSION TOOL

- Fast and accurate conversions
- Simplifies complex binary representation
- Useful in programming and digital systems

How to Convert Hex to Binary

Lógica: Conversor no **MARS**

- **Isolamento de Bits:** Usamos uma máscara de bits binária (operação lógica AND `0x0000000F`) para isolar unicamente os 4 bits menos significativos.
- **Deslocamento Dinâmico (Shift):** Aplicamos `sr1` (Shift Right Logical) em 4 bits para mover o próximo grupo à direita para a posição ativa.
- **Tratamento Condicional:**
 - → Se nibble < 10 : Adiciona 48.
 - → Se nibble ≥ 10 : Adiciona 55.
- **Saída:** Invoca `syscall 11` para imprimir o caractere ASCII montado.

CONVERSÃO DE DÍGITO (MIPS)

```
# Processa nibble em $t1
andi $t1, $t0, 15      # Máscara 0x0F (último dígito)
slti $t2, $t1, 10      # Verifica se valor < 10
beq  $t2, $zero, letra # Se falso, desvia p/ letra

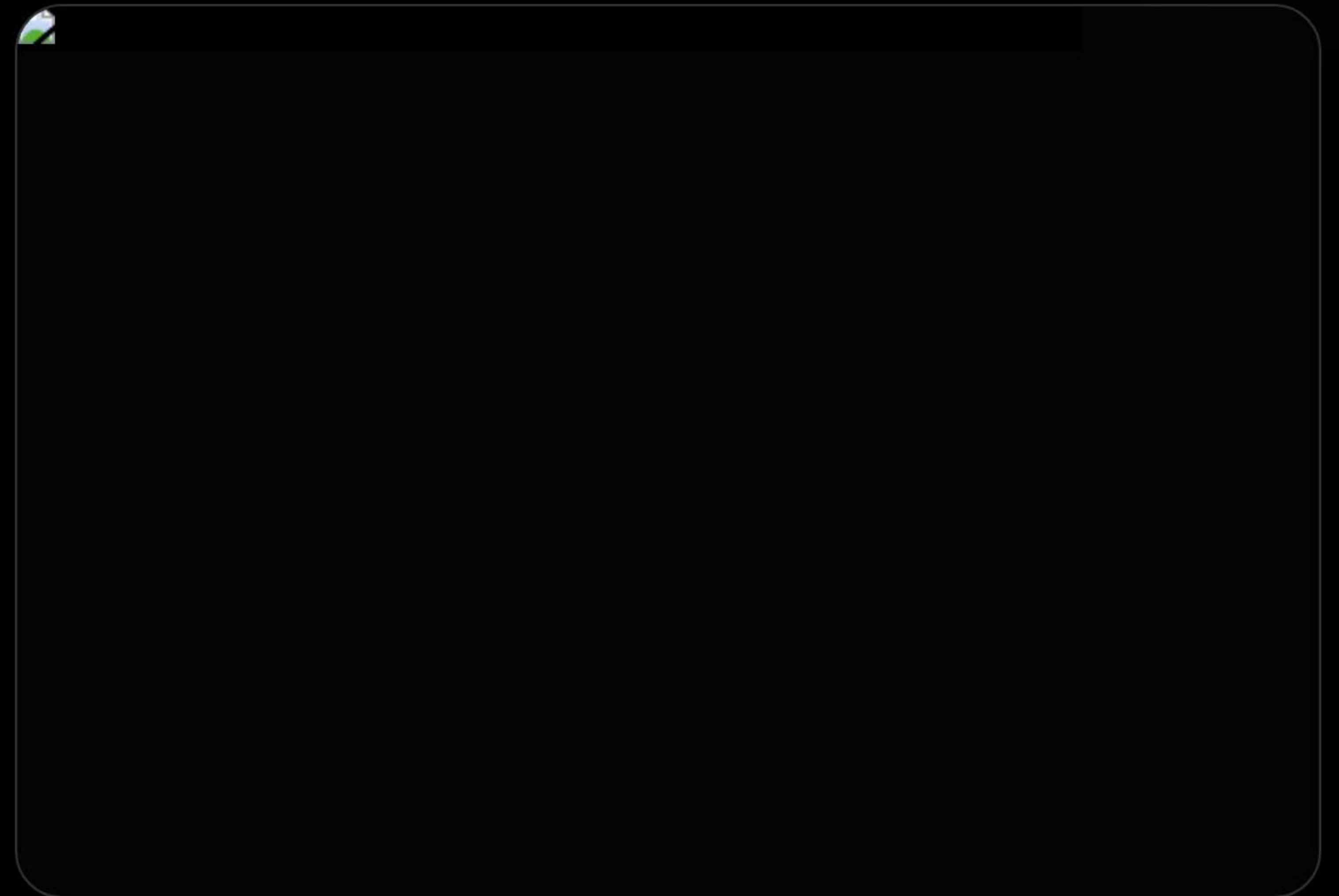
addi $a0, $t1, 48      # Converte num: valor + '0'
j    print

letra:
    addi $a0, $t1, 55 # Converte letra: valor + 55
print:
    li   $v0, 11      # Serviço syscall: Print Char
    syscall
```


Teoria: Ordenação de Vetores

A ordenação em memória ramifica em complexidade algorítmica. O **Bubble Sort** trabalha varrendo o vetor continuamente.

- **Mecânica Operacional:** Compara pares adjacentes e realiza a troca física (swap) se estiverem fora da ordenação desejada.
- **In-Place e Eficiência:** Possui complexidade $O(n^2)$, mas sua execução é classificada como "In-place" ($O(1)$ espaço auxiliar), o que evita alocações extras no heap do hardware.
- **Importância no Assembly:** Ótimo laboratório para compreender aritmética pura de endereçamento físico e leitura/escrita em barramentos.



Lógica: Ordenação no **MARS**

- **Aritmética de Ponteiros:** O MIPS endereça em nível de byte. Como cada palavra (word) consome 4 bytes, os índices do vetor avançam via `addi $a0, $a0, 4`.
- **Troca na Memória (Swap):**
 - Carrega elementos para registradores: `lw $t0, 0($a0)` e `lw $t1, 4($a0)`.
 - Compara e, se necessário, salva invertido na RAM: `sw $t1, 0($a0)` e `sw $t0, 4($a0)`.

ROTINA DE SWAP MIPS

```
# $a0 contém o endereço do índice atual 'i'
lw  $t0, 0($a0)      # $t0 = v[i]
lw  $t1, 4($a0)      # $t1 = v[i+1]

slt  $t2, $t1, $t0    # Verifica se v[i+1] < v[i]
beq  $t2, $zero, skip # Se não for menor, ignora swap

# Se menor, realiza troca física
sw  $t1, 0($a0)      # v[i] = $t1
sw  $t0, 4($a0)      # v[i+1] = $t0

skip:
```

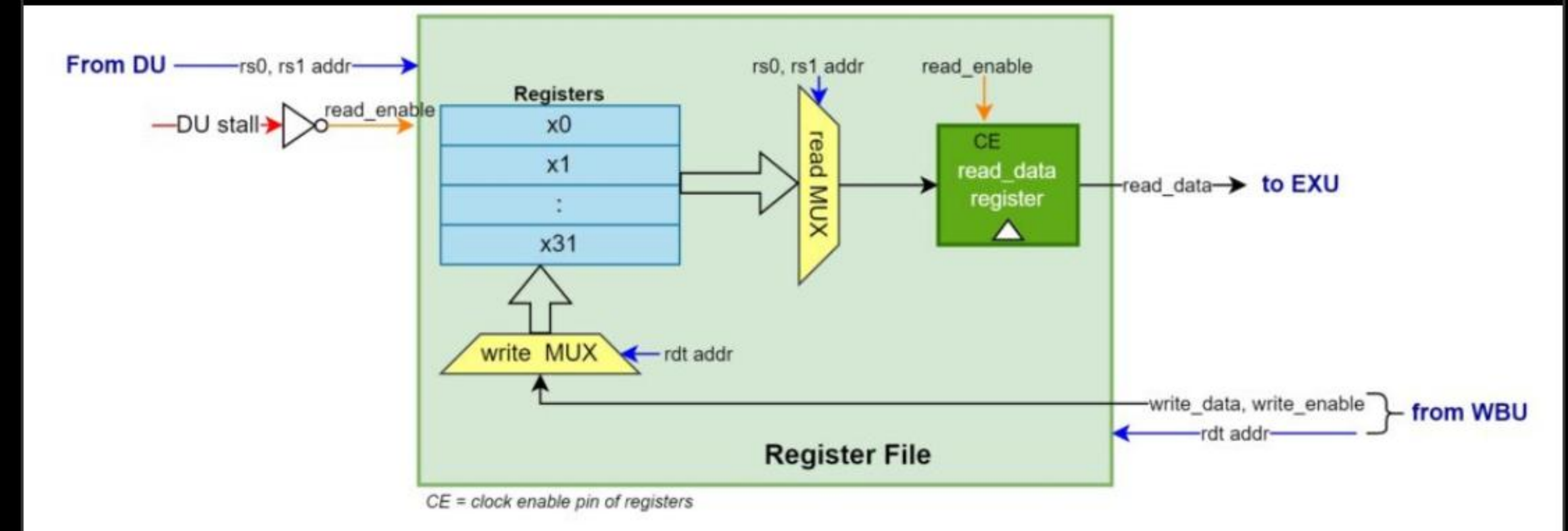
Parte 2: **A Era RISC-V**

Migração e Tecnologias Modernas

O Ecossistema RISC-V

O RISC-V é uma arquitetura de conjunto de instruções (ISA) aberta, projetada para superar as limitações comerciais e de patentes de modelos clássicos.

- **Arquitetura Modular:** Define um conjunto básico fixo mínimo (ex: RV32I para inteiros de 32 bits) ao qual se acoplam extensões padronizadas opcionais como M (Multiplicação), F (Precisão Simples) e A (Atômicas).
- **Adopção Massiva:** Por ser livre de royalties, tornou-se o padrão unificado de pesquisa acadêmica e o núcleo de soluções corporativas de hardware.



Comparação: MIPS vs RISC-V

Característica Técnica	MIPS (MARS)	RISC-V (RARS)
Chamada de Sistema	Instrução <code>syscall</code>	Instrução <code>ecall</code>
Reg. Código de Serviço	Registrador <code>\$v0</code> (ou <code>\$2</code>)	Registrador <code>a7</code> (ou <code>x17</code>)
Registrador de Retorno	Registrador <code>\$v0</code>	Registrador <code>a0</code>
Nomes da ABI	<code>\$zero</code> , <code>\$t0</code> , <code>\$s0</code> , <code>\$a0...</code>	<code>x0</code> , <code>t0</code> , <code>s0</code> , <code>a0...</code> (Mapeados na ABI)
Instruções de Desvio	Requer flag intermediária (ex: <code>slt + beq</code>)	Comparações diretas avançadas (ex: <code>blt</code> , <code>bge</code>)